Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина: Модели данных и системы управления базами данных

К защите допустить:

И.О. Заведующего кафедрой информатики
_____С. И. Сиротко

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
на тему

ИНФОРМАЦИОННАЯ СИСТЕМА КОЛЛЕКТИВНОГО РЕДАКТИРОВАНИЯ СТАТЕЙ ОБ ИГРОВЫХ ВСЕЛЕННЫХ

БГУИР КП 6-05 0612 02 017 ПЗ

Студент Д. А. Левшуков

Руководитель И. Г. Скиба

Минск 2026

СОДЕРЖАНИЕ

Введение 5
1 Обзор существующих аналогов 8
1.1 Анализ предметной области 8
1.2 Анализ существующих информационных систем 9
1.3 Формирование требований к разрабатываемой системе 11
2 Функциональные требования и инструменты разработки 8
2.1 Функциональные и нефункциональные требования 8
2.2 Выбор архитектуры информационной системы 8
2.3 Выбор языка программирования и фреймворка 8
2.4 Выбор системы управления базами данных 8
2.5 Выбор средств кэширования и хранения мультимедийных данных 8
3 Проектирование информационной системы 8
3.1 Разработка структуры информационной системы 8
3.2 Разработка концептуальной модели 8
3.3 Разработка логической модели данных 8
3.4 Разработка физической модели базы данных 8
3.5 Проектирование API и взаимодействия сервисов 8
3.6 Проектирование пользовательских сценариев и обработки данных 8
4 Разработка информационной системы 8
4.1 Реализация базы данных 8
4.2 Реализация серверной части 8
4.3 Реализация аутентификации и разграничения доступа 8
4.4 Реализация механизма редактирования и версионирования статей 8
4.5 Реализация работы с Redis 8
4.6 Реализация работы с мультимедийными материалами 8
5 Тестирование информационной системы 8
5.1 Разработка сценариев тестирования 8
5.2 Тестирование основных функций 8
5.3 Тестирование базы данных 8
5.4 Тестирование механизмов версионирования и разграничения доступа 8
5.5 Анализ результатов тестирования 8
6 Руководство по развёртыванию приложения 8
6.1 Требования к программному и аппартному обеспечению 8
6.2 Настройка окружения и конфигурации системы 8
6.3 Развёртывание и запуск информационной системы 8
6.4 Проверка работоспособности после запуска 8
7 Руководство пользователя 8
7.1 Начало работы с системой 8
7.2 Регистрация и аутентификация пользователей 8
7.3 Навигация по игровым вселенным и поиск статей 8
7.4 Создание и редактирование статей 8
7.5 Работа с комментариями и жалобами 8

7.6 Просмотр уведомлений и управление профилем 8
Заключение 34
Список литературных источников 35
ПРИЛОЖЕНИЕ А 36
ПРИЛОЖЕНИЕ Б (обязательное) Листинг программного кода 37
ПРИЛОЖЕНИЕ В (обязательное) Концептуальная модель 42
ПРИЛОЖЕНИЕ Г (обязательное) Логическая модель 43
ПРИЛОЖЕНИЕ Д (обязательное) Физическая модель базы данных 44
ПРИЛОЖЕНИЕ Е (обязательное) Ведомость документов 45

ВВЕДЕНИЕ

Компьютерные игры формируют обширные вымышленные миры, включающие персонажей, локации, предметы, организации и события. По мере развития игровых серий объём сведений об этих объектах увеличивается, а информация уточняется участниками тематических сообществ. Это создаёт потребность в информационных системах, обеспечивающих структурированное хранение материалов, совместное редактирование статей, сохранение истории изменений и поиск информации.

Использование разрозненных веб-страниц затрудняет установление связей между материалами, контроль изменений и поддержание актуальности сведений. Централизованная информационная система позволяет объединить статьи в иерархическую структуру, поддерживать переводы на разные языки и организовать совместную работу пользователей.

Особенности предметной области определяют требования к целостности связей между статьями, сохранению истории редакций, поддержке нескольких языков и возможности перемещения материалов в иерархии. Совместное редактирование требует предварительной модерации изменений и проверки версии материала, на основе которой создано предложение. Также необходимы разграничение прав доступа, поиск по опубликованным статьям и безопасное хранение изображений.

Цель курсового проекта – разработка информационной системы коллективного редактирования статей об игровых вселенных.

Для достижения поставленной цели необходимо решить следующие задачи:

1 Проанализировать предметную область и существующие системы для хранения и совместного редактирования энциклопедических материалов.

2 Сформировать функциональные и нефункциональные требования к разрабатываемой системе, определить входные и выходные данные основных операций.

3 Разработать модель предметной области, выделить основные сущности и определить связи между ними.

4 Спроектировать реляционные базы данных отдельных сервисов с учётом требований нормализации, целостности и независимости хранения данных.

5 Разработать серверные компоненты для управления пользователями, статьями, комментариями, жалобами и уведомлениями.

6 Реализовать механизм версионирования статей, обработки предложений изменений и их модерации.

7 Организовать полнотекстовый поиск по опубликованным версиям статей и иерархической структуре материалов.

8 Реализовать хранение изображений в объектном хранилище и взаимодействие сервисов посредством HTTP-запросов и событий.

9 Предусмотреть механизмы поддержания целостности данных в PostgreSQL, включая триггеры и хранимые процедуры.

10 Провести тестирование основных сценариев работы системы, включая аутентификацию, редактирование статей, модерацию, поиск, управление доступом и загрузку файлов.

Для реализации серверной части используются C++20 и фреймворк Drogon. Структурированные данные хранятся в PostgreSQL, токены обновления и список заблокированных пользователей – в Redis. Для обмена событиями между сервисами применяется RabbitMQ, для хранения изображений – MinIO. Развёртывание компонентов организуется с помощью Docker Compose. Серверная часть разделена на специализированные сервисы, использующие собственные базы данных или схемы без прямых связей с хранилищами других сервисов.

Практическая значимость работы заключается в создании программной основы для тематических энциклопедий о компьютерных играх. Система объединяет структурированное хранение статей, управление версиями, совместное редактирование и модерацию предложений. Разграничение прав доступа и ведение журнала действий позволяют контролировать операции пользователей, а взаимодействие сервисов обеспечивает выполнение связанных функций системы.

Пояснительная записка оформлена в соответствии с СТП 01-2024. [1]

Справка о проверке на заимствование приведена в приложении А.

1 ОБЗОР СУЩЕСТВУЮЩИХ АНАЛОГОВ

Предметная область включает создание, хранение и коллективное редактирование материалов об игровых вселенных. При проектировании системы необходимо учитывать иерархические связи между статьями, поддержку переводов, сохранение истории редакций, модерацию изменений, поиск информации и разграничение прав доступа. Анализ существующих решений позволяет определить требования к разрабатываемой системе и обосновать выбор её архитектуры.

1.1 Анализ предметной области

Предметную область курсового проекта составляет создание, хранение и коллективное редактирование материалов об игровых вселенных. Под игровой вселенной понимается совокупность взаимосвязанных объектов и событий, относящихся к определённой игре или серии игр. К таким объектам относятся персонажи, локации, предметы, организации и сюжетные события.

Основным информационным объектом является статья. Она содержит заголовок и текстовое описание объекта, а также может включать внутренние ссылки на другие статьи и изображения. Материалы организуются в виде иерархического дерева: корневые узлы соответствуют игровым вселенным, а дочерние узлы описывают отдельные элементы. Каждый узел может иметь дочерние элементы, что позволяет формировать структуру произвольной глубины.

Для каждой статьи предусматривается поддержка переводов. Содержимое и история изменений хранятся отдельно для каждого языка. Если перевод на выбранный язык отсутствует, система должна сообщить об этом и предоставить список доступных языков. Такая организация позволяет поддерживать многоязычность без создания независимых копий всей структуры материалов.

Важной особенностью предметной области является коллективное редактирование. Несколько пользователей могут предлагать изменения одной статьи, поэтому система должна сохранять предыдущие редакции и учитывать версию, на основе которой подготовлено каждое предложение. Перед публикацией изменения проходят модерацию. Если базовая версия статьи успела измениться, предложение признаётся устаревшим и должно быть подготовлено повторно. Это предотвращает применение изменений, основанных на неактуальном содержимом.

Работа с большим количеством материалов требует средств навигации и поиска. Пользователю необходимо получать сведения о родительских узлах, просматривать дочерние элементы и переходить по внутренним ссылкам. Для поиска используется полнотекстовый механизм PostgreSQL, работающий с опубликованными актуальными версиями статей. Поиск может выполняться по всей базе знаний или в пределах определённой ветви дерева [2].

В содержимом статей используются изображения, иллюстрирующие персонажей, локации и другие объекты. Для хранения таких файлов целесообразно применять отдельное объектное хранилище, а в базе данных сохранять идентификаторы файлов, их характеристики и сведения о принадлежности к статье или предложению изменения. Загруженные изображения, относящиеся к предложению, остаются в закрытой области до принятия решения модератором. После одобрения они становятся общедоступными, а после отклонения удаляются.

Система должна поддерживать пользователей с различными уровнями полномочий: обычных пользователей, модераторов, администраторов и суперпользователя. Незарегистрированные посетители могут просматривать опубликованные материалы и выполнять поиск. Авторизованные пользователи могут предлагать изменения и работать с комментариями и жалобами. Модераторы рассматривают предложения и жалобы, а административные роли дополнительно управляют пользователями и их полномочиями. Действия пользователей, имеющие значение для контроля работы системы, фиксируются в журнале аудита.

Таким образом, предметная область характеризуется иерархической структурой материалов, наличием переводов, необходимостью сохранения истории редакций и обязательной модерацией пользовательских изменений. Эти особенности определяют требования к модели данных, механизмам поиска, управлению доступом и организации взаимодействия компонентов системы.

1.2 Анализ существующих информационных систем

В качестве первого аналога рассматривается Fandom – платформа тематических wiki, включающих большое количество баз знаний по компьютерным играм и другим направлениям. Ранее игровое направление было представлено, в частности, платформой Gamepedia, впоследствии объединённой с Fandom [3].

Основу организации информации составляет wiki-модель: материалы размещаются на отдельных страницах, связываются внутренними ссылками и могут редактироваться участниками сообщества. Для игровых проектов такой подход позволяет объединять сведения о персонажах, локациях, предметах и событиях, обеспечивая переходы между связанными статьями.

К достоинствам Fandom относятся поддержка коллективного редактирования, развитые средства организации материалов и ориентация на крупные сообщества. Вместе с тем платформа представляет собой готовый универсальный сервис, а не учебную основу для самостоятельной разработки серверной архитектуры и базы данных. Поэтому её использование не позволяет в рамках данного курсового проекта в полной мере продемонстрировать проектирование отдельных сервисов и реализацию механизмов обработки данных на уровне PostgreSQL. На рисунке 1.1 представлен интерфейс Fandom.

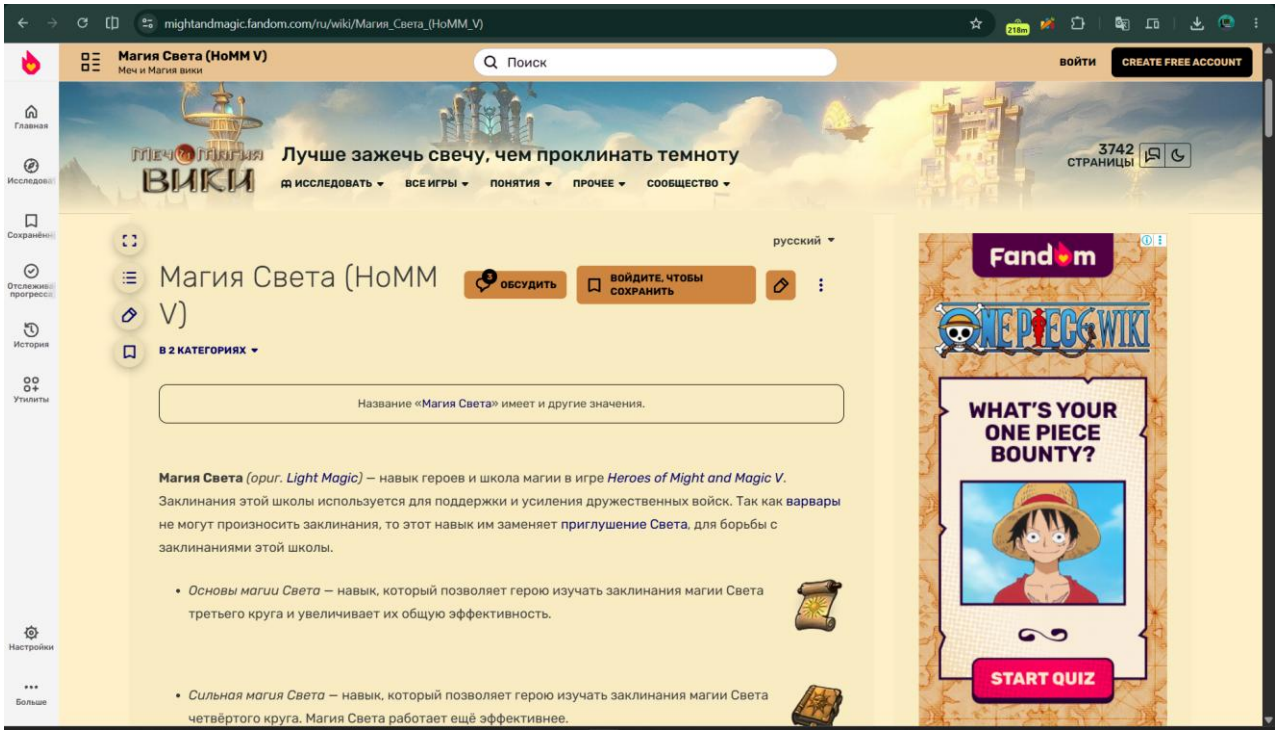


Рисунок 1.1 – Интерфейс Fandom

Вторым аналогом является wiki.gg – платформа, ориентированная преимущественно на игровые сообщества и тематические базы знаний [4]. Как и в Fandom, информация организуется в виде страниц, объединённых общей тематикой. Пользователи могут совместно создавать и редактировать материалы, посвящённые различным аспектам игровых проектов.

К преимуществам wiki.gg относится специализация на игровых сообществах. Такая направленность соответствует рассматриваемой предметной области и позволяет использовать платформу в качестве примера организации тематической базы знаний. Однако для курсового проекта важно не только предоставить пользователям готовую wiki, но и самостоятельно реализовать механизмы хранения данных, управления редакциями, модерации и взаимодействия серверных компонентов. На рисунке 1.2 представлен интерфейс wiki.gg.

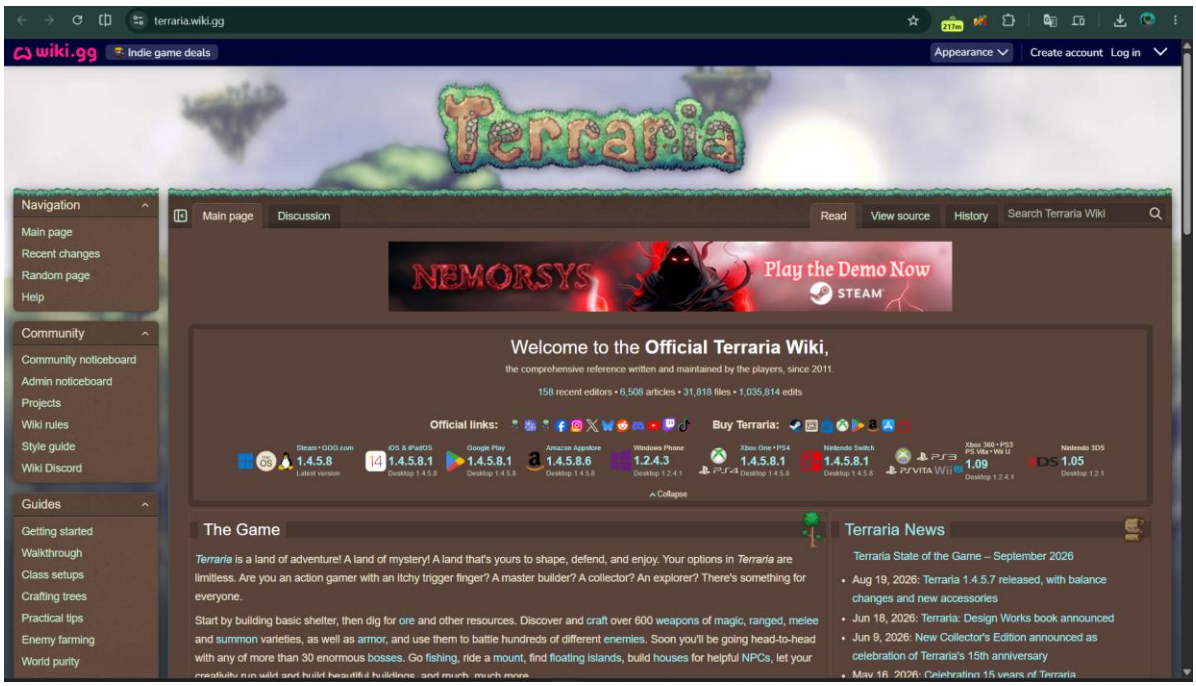


Рисунок 1.2 – Интерфейс wiki.gg

Третьим аналогом является Miraheze – платформа для размещения wiki различной тематики [5]. Она предоставляет инфраструктуру для создания отдельных информационных пространств и расширения их возможностей средствами платформы MediaWiki.

Достоинством Miraheze является возможность адаптации wiki под различные задачи. Вместе с тем использование готовой платформы ограничивает практическую ценность самостоятельной разработки серверной части и физической модели базы данных в рамках рассматриваемого курсового проекта. На рисунке 1.3 представлен интерфейс Miraheze.

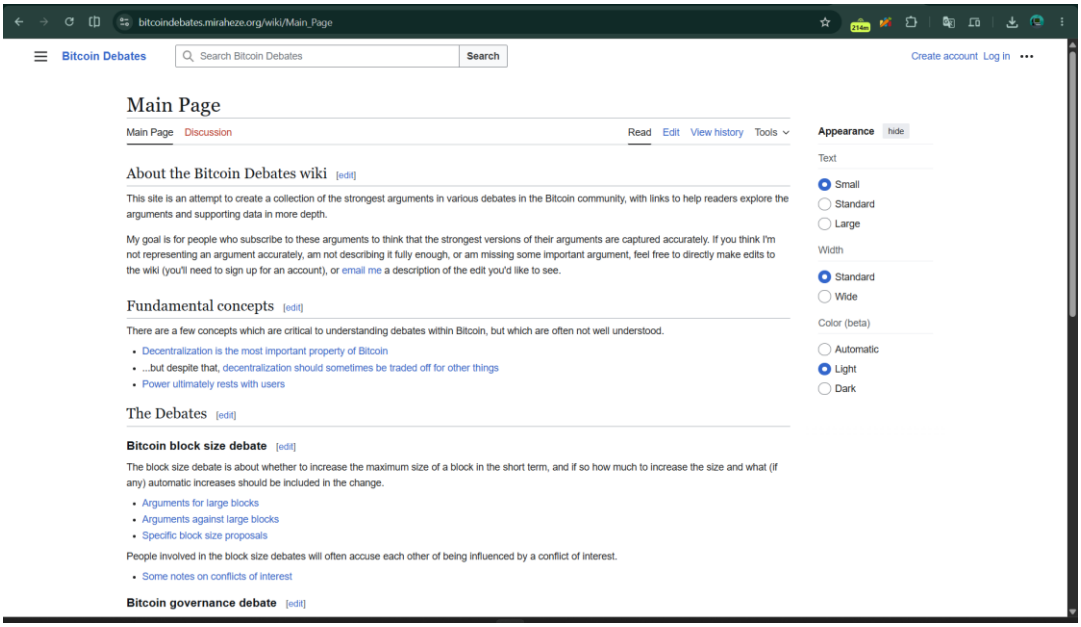


Рисунок 1.3 – Интерфейс Miraheze

Все три аналога обеспечивают основные возможности wiki-систем: публикацию статей, внутреннюю навигацию и коллективное формирование содержимого. Отличия связаны с направленностью платформ и предоставляемыми средствами настройки. При разработке собственной системы целесообразно использовать общие принципы wiki-модели, но реализовать необходимые механизмы самостоятельно.

Разрабатываемая система ориентирована на компактное решение с определённым набором функций: иерархическую организацию материалов, поддержку переводов, хранение истории редакций, обязательную модерацию изменений, полнотекстовый поиск, комментарии, жалобы, уведомления и управление полномочиями пользователей. Архитектура предусматривает разделение серверной логики на отдельные сервисы и использование самостоятельной базы данных для каждого сервиса.

Таким образом, анализ аналогов позволяет определить основные принципы организации информационной системы, при этом собственная разработка ориентирована на изучение и реализацию механизмов хранения, обработки и защиты данных, а не на воспроизведение всего функционального объёма существующих платформ.

1.3 Формирование требований к разрабатываемой системе

На основании анализа предметной области и существующих аналогов формируются требования к информационной системе коллективного редактирования статей об игровых вселенных.

Назначение системы – предоставление средств для хранения, поиска и коллективного редактирования статей об игровых вселенных с сохранением истории изменений и контролем публикации пользовательских предложений.

Система должна выполнять следующие основные функции:

1 Предоставлять доступ к опубликованным статьям, сведениям об их родительских и дочерних узлах, доступных языках и истории редакций.

2 Обеспечивать регистрацию пользователей, аутентификацию, завершение сеанса и обновление токенов доступа.

3 Позволять авторизованным пользователям создавать предложения на добавление, изменение, перемещение и удаление материалов.

4 Сохранять базовую версию статьи, связанную с предложением изменения, и определять устаревшие предложения при изменении актуальной версии.

5 Предоставлять модераторам и администраторам очередь предложений, возможность их рассмотрения и принятия решения.

6 Обеспечивать полнотекстовый поиск по опубликованным актуальным версиям статей с возможностью ограничения поиска определённой ветвью дерева.

7 Поддерживать создание комментариев и ответов, редактирование собственных комментариев и удаление комментариев в соответствии с полномочиями пользователя.

8 Обеспечивать создание жалоб, просмотр их статусов и рассмотрение модераторами.

9 Предоставлять уведомления о значимых событиях, включая решения по предложениям и жалобам, блокировку пользователя и ответы на комментарии.

10 Обеспечивать загрузку и хранение изображений, связанных со статьями и предложениями изменений.

11 Поддерживать управление ролями и блокировками пользователей, а также ведение журнала аудита.

12 Обеспечивать разграничение доступа к операциям в соответствии с ролью и полномочиями пользователя.

К входным данным относятся учётные данные при регистрации и аутентификации, сведения профиля, заголовки и текст статей, выбранный язык, идентификаторы базовых редакций, параметры перемещения узлов, текст комментариев и жалоб, поисковые запросы, идентификаторы объектов и загружаемые файлы. Для операций модерации входными данными являются идентификатор рассматриваемого предложения или жалобы и решение уполномоченного пользователя.

К выходным данным относятся опубликованные статьи и их переводы, сведения о структуре материалов, результаты полнотекстового поиска, история редакций, результаты сравнения версий, статусы предложений и жалоб, списки комментариев и ответов, уведомления и сведения о выполненных действиях. При аутентификации система возвращает токены доступа и обновления. При операциях, связанных с файлами, возвращаются необходимые сведения для загрузки или использования изображения.

К требованиям надёжности относятся сохранение истории редакций, защита от применения устаревших изменений, обеспечение целостности данных и обработка повторно доставленных событий без дублирования результатов. Обмен событиями между сервисами должен использовать устойчивые очереди и сообщения с подтверждением после записи данных в базу. Обработчики событий должны поддерживать идемпотентность.

К требованиям безопасности относятся хранение паролей в виде хешей, проверка токенов доступа, разграничение полномочий и немедленная блокировка запросов заблокированного пользователя на уровне шлюза. Пользователь не должен иметь возможности изменять данные или выполнять административные действия без соответствующих прав.

Система должна работать в среде, поддерживающей Docker Compose, PostgreSQL, Redis, RabbitMQ и MinIO. Серверные сервисы разрабатываются на C++20 с использованием фреймворка Drogon. Взаимодействие с базами данных осуществляется через интерфейс DbClient с использованием SQL-запросов без ORM. Для обмена событиями применяется RabbitMQ, для синхронных запросов между компонентами – HTTP, для доставки уведомлений в реальном времени – Server-Sent Events (SSE).

Базы данных отдельных сервисов должны быть разделены и не должны содержать прямых связей между собой. Структура каждой базы данных

должна соответствовать требованиям третьей нормальной формы, если особенности конкретных данных не требуют иного обоснованного решения. Для поддержания целостности и автоматизации операций применяются ограничения, индексы, триггеры и хранимые процедуры PostgreSQL.

Таким образом, сформированные требования определяют состав функций, входные и выходные данные, основные ограничения безопасности и надёжности, а также технические средства реализации. На их основе выполняются дальнейшее проектирование архитектуры приложения, разработка моделей баз данных и детализация алгоритмов обработки данных.

2 ФУНКЦИОНАЛЬНЫЕ ТРЕБОВАНИЯ И ИНСТРУМЕНТЫ РАЗРАБОТКИ

На основании проведённого анализа предметной области и существующих информационных систем необходимо определить функциональные и нефункциональные требования к разрабатываемой системе, а также обосновать выбор архитектуры и программных средств. Основное внимание уделяется организации хранения и обработки информации, ведению истории изменений, модерации пользовательских предложений и разграничению доступа.

Разрабатываемая информационная система предназначена для коллективного формирования базы знаний об игровых вселенных. Она должна обеспечивать просмотр и поиск материалов, создание и редактирование статей, обсуждение содержимого, отправку жалоб и управление пользовательскими правами. Для хранения структурированных данных используется PostgreSQL, для отдельных вспомогательных задач – Redis, для обмена событиями между сервисами – RabbitMQ, а для хранения изображений – объектное хранилище MinIO.

Серверная часть организуется в виде набора взаимодействующих микросервисов. Каждый сервис отвечает за определённую функциональную область и использует собственные данные. Взаимодействие с клиентским приложением осуществляется через API Gateway, который маршрутизирует запросы и выполняет предусмотренные архитектурой проверки доступа.

2.1 Функциональные и нефункциональные требования

Назначением разрабатываемой информационной системы является предоставление единого информационного пространства для хранения, поиска и коллективного редактирования статей об игровых вселенных. Система должна поддерживать иерархическую организацию материалов, хранение переводов, историю редакций, обсуждение статей и модерацию пользовательских изменений.

Основной функцией системы является работа со статьями. Материалы организуются в виде дерева, корневыми узлами которого выступают игровые вселенные. Каждый узел может содержать дочерние элементы и одновременно выступать родителем для других материалов. Узлы верхнего уровня предназначены преимущественно для общего описания, а конечные узлы – для более детальной информации. Для каждого узла необходимо хранить идентификатор, уникальный в соответствующем контексте идентификатор-строку (slug), путь в дереве и сведения об удалении.

Система должна поддерживать создание, редактирование, перемещение и удаление узлов. Пользовательские операции изменения материалов выполняются через механизм предложений и проходят модерацию. Перемещение узла не должно приводить к образованию циклической структуры: узел нельзя переместить внутрь собственного поддерева. Удаление

допускается только для узлов, у которых отсутствуют дочерние элементы. Для удаления применяется мягкое удаление с сохранением соответствующей записи и отметки времени.

Важной функцией является поддержка переводов. Один узел может иметь версии содержимого на нескольких языках. Для каждого языка отдельно сохраняются название, текст и история редакций. Если перевод на запрошенный язык отсутствует, система должна сообщить об этом и предоставить информацию о доступных языках.

Система должна обеспечивать коллективное редактирование материалов. Пользователь отправляет предложение, содержащее полный новый текст статьи. Предложение сохраняет ссылку на версию, на основе которой оно было подготовлено. Перед применением изменения система проверяет, является ли эта версия актуальной. Если материал уже изменился, предложение признаётся устаревшим, а пользователь получает соответствующее уведомление. В таком случае необходимо подготовить и отправить новое предложение.

До принятия решения предложение доступно его автору, а также модераторам и администраторам, обладающим соответствующими полномочиями. Модератор рассматривает предложения других пользователей. Администратор и супер администратор могут рассматривать и собственные предложения; такие действия фиксируются в журнале аудита. Предложения обрабатываются в порядке очереди. После одобрения создаётся новая версия материала, а остальные ожидающие предложения для того же перевода признаются устаревшими.

Для анализа изменений система должна формировать построчное сравнение исходного и предлагаемого текстов. Это позволяет модератору определить добавленные, изменённые и удалённые фрагменты. История версий должна сохранять сведения о содержимом, авторе и времени создания редакции.

Взаимосвязи между материалами организуются с помощью двух механизмов. Первый – иерархическая структура узлов, которая обеспечивает навигацию по игровым вселенным, родительским элементам и поддеревьям. Второй – внутренние ссылки, указываемые непосредственно в тексте статьи в формате [[...]]. Отдельная система категорий и самостоятельная таблица связей между статьями не предусматриваются.

Одной из функций системы является полнотекстовый поиск. Пользователь должен иметь возможность вводить запрос и получать опубликованные материалы, соответствующие его содержанию. Поиск выполняется по актуальным опубликованным версиям статей. При необходимости область поиска может ограничиваться определённой ветвью дерева. Для реализации поиска используются встроенные средства PostgreSQL, включая полнотекстовые индексы и формирование фрагментов текста с выделением совпадений. Конфигурация поиска учитывает язык перевода [6].

Система должна поддерживать добавление изображений к пользовательским изменениям. Изображения являются частью предложения об изменении статьи и проходят модерацию вместе с ним. Для каждого файла необходимо хранить метаданные, включая ключ объекта в хранилище, размер, формат и сведения о связанном предложении или опубликованном переводе. Сами файлы размещаются в MinIO, а в PostgreSQL сохраняются сведения, необходимые для управления ими.

При загрузке изображения проверяются его фактический формат и допустимый размер. Поддерживаются форматы PNG, JPEG и WebP. Архитектурой предусмотрены ограничения на размер одного файла, общий объём изображений в предложении и их количество. Загрузка выполняется с использованием предварительно подписанных ссылок. Изображения ожидающих рассмотрения предложений доступны автору и модераторам; после одобрения они переводятся в опубликованное хранилище, а при отклонении удаляются. Для очистки временных файлов предусмотрена фоновая задача.

Система должна поддерживать регистрацию, аутентификацию и управление пользовательскими правами. Предусмотрены роли супер администратора, администратора, модератора и пользователя. Доступность административных операций определяется ролью и установленными полномочиями. Назначение и снятие ролей выполняется в соответствии с иерархией полномочий. Для заблокированных пользователей применяется запрет на вход в систему, чтение и изменение данных.

Для защиты учётных записей необходимо хранить пароли в виде хешей, а не в открытом виде. Для аутентификации используются токены доступа и обновления. Токены обновления хранятся в Redis, а их ротация позволяет обнаруживать повторное использование ранее выданного токена. Сведения о блокировках также хранятся в Redis и проверяются API Gateway при обработке запросов. Ограничение частоты запросов используется как дополнительная мера защиты от чрезмерной нагрузки и автоматизированных запросов.

К функциональным требованиям относятся работа с комментариями и жалобами. Пользователь должен иметь возможность добавлять комментарии к переводу статьи и отвечать на существующие комментарии. Вложенность ответов не ограничивается фиксированным уровнем. Пользователь может редактировать и удалять собственные комментарии, а модератор имеет право удалять комментарии других пользователей. Для удаления комментариев применяется мягкое удаление.

Система должна обеспечивать отправку жалоб на материалы и комментарии, просмотр пользователем собственных жалоб и обработку жалоб модераторами. Запрос на удаление узла также оформляется как специальный тип жалобы с обязательным указанием причины. После одобрения такого запроса сервис статей выполняет предусмотренную операцию удаления.

Необходимо реализовать механизм уведомлений. Пользователи должны получать уведомления о значимых событиях, например об одобрении,

отклонении или устаревании предложения, разрешении жалобы, блокировке учётной записи и ответах на комментарии. Система должна предоставлять список уведомлений и операции управления их состоянием. Для доставки уведомлений в реальном времени используется SSE-соединение, проходящее через API Gateway. При повторном подключении система должна иметь возможность передать уведомления, пропущенные во время разрыва соединения.

Дополнительным функциональным требованием является ведение журнала аудита. В нём фиксируются значимые действия пользователей и административные операции, в том числе регистрация, вход, изменение профиля, отправка и рассмотрение предложений, создание и удаление комментариев, обработка жалоб, блокировки и изменение ролей. Просмотр записей журнала ограничивается правами соответствующей роли. Чтение статей само по себе не относится к действиям, которые должны фиксироваться в журнале аудита. Срок хранения записей составляет 180 дней и может настраиваться.

Входными данными системы являются сведения для регистрации и аутентификации, параметры пользовательского профиля, названия и тексты материалов, языки переводов, идентификаторы узлов, данные предложений об изменении, комментарии, жалобы, поисковые запросы и изображения. Для административных операций входными данными выступают параметры изменения ролей, блокировок и решений по предложениям и жалобам.

Выходными данными являются страницы статей, сведения о доступных переводах, результаты поиска, история редакций, сравнение версий, комментарии, статусы предложений и жалоб, уведомления и сведения о выполненных операциях. При невозможности выполнить запрос система должна возвращать соответствующую информацию об ошибке, например об отсутствии прав, несуществующем объекте или устаревшей версии предложения.

Нефункциональные требования определяют характеристики надёжности, безопасности, производительности и расширяемости системы. Операции с несколькими связанными записями должны выполняться согласованно. Для этого используются транзакции и ограничения целостности PostgreSQL. При обработке событий необходимо исключить повторное применение одного и того же события, а сообщения должны подтверждаться после успешной записи необходимых данных.

Производительность обеспечивается использованием индексов PostgreSQL, полнотекстового поиска, Redis для предусмотренных архитектурой задач и разделением ответственности между сервисами. Частота обращений к отдельным компонентам должна контролироваться, а длительные операции с изображениями не должны требовать передачи всего бинарного содержимого через основной поток обработки запросов, если для этого предусмотрена загрузка по предварительно подписанным ссылкам.

Безопасность обеспечивается проверкой токенов доступа, проверкой ролей и полномочий, обработкой списка блокировок, ограничением частоты

запросов и хранением паролей в виде хешей. Клиентское приложение не получает непосредственного доступа к базам данных. При эксплуатации системы необходимо использовать защищённые сетевые соединения.

Расширяемость обеспечивается разделением серверной части на самостоятельные микросервисы. Каждый сервис владеет своими данными и предоставляет другим компонентам доступ к ним через API или предусмотренный механизм обмена событиями. Это позволяет изменять отдельные функциональные компоненты без необходимости объединять их данные в общую базу.

Условием эксплуатации системы является наличие серверной инфраструктуры, поддерживающей запуск микросервисов, PostgreSQL, Redis, RabbitMQ и MinIO. Для развёртывания компонентов используется Docker Compose. Клиентская часть взаимодействует с серверной через API Gateway, а внутренние сервисы обращаются друг к другу посредством определённых интерфейсов.

Таким образом, функциональные требования определяют основные операции с материалами, переводами, предложениями, комментариями, жалобами и уведомлениями. Нефункциональные требования устанавливают ограничения по безопасности, надёжности, производительности и расширяемости. Эти требования служат основой для выбора архитектуры и программных средств реализации системы.

2.2 Выбор архитектуры информационной системы

Для разрабатываемой информационной системы целесообразно использовать клиент-серверную архитектуру. Клиентская часть отвечает за взаимодействие с пользователем и отображение информации, а серверная часть выполняет обработку запросов, проверку полномочий, реализацию прикладной логики и взаимодействие с хранилищами данных.

Непосредственное подключение клиентского приложения к PostgreSQL не предусматривается. Клиент не должен получать доступ к структуре базы данных и выполнять запросы к ней напрямую. Проверка полномочий и обработка пользовательских операций выполняются на серверной стороне, которая рассматривается как доверенная часть системы.

Для организации серверной части выбрана микросервисная архитектура. Приложение разделяется на несколько самостоятельных сервисов, каждый из которых отвечает за отдельную функциональную область. В проекте выделяются API Gateway, User Service, Article Service, Comment Service, Report Service и Notification Service.

API Gateway является единой точкой входа для клиентских запросов. Он маршрутизирует запросы к соответствующим сервисам, проверяет токены доступа, проверяет наличие пользователя в списке блокировок Redis и ограничивает частоту запросов. При передаче запроса внутреннему сервису Gateway передаёт идентификатор пользователя и сведения о его роли. Через Gateway также организуется передача уведомлений по SSE. При блокировке

пользователя или истечении срока действия токена соответствующее соединение с потоком уведомлений закрывается.

User Service отвечает за регистрацию и аутентификацию, управление ролями и блокировками, профилями и аватарами пользователей. Этот сервис владеет данными учётных записей и журналом аудита. Он также обрабатывает поступающие события действий других сервисов и сохраняет соответствующие записи аудита.

Article Service отвечает за узлы дерева материалов, переводы, версии статей, предложения об изменении, модерацию, полнотекстовый поиск и изображения статей. В этом сервисе реализуются операции перемещения узлов, проверки актуальности базовой версии предложения и применения одобренных изменений.

Comment Service отвечает за комментарии и ответы, а также за редактирование и удаление комментариев в соответствии с правами пользователя. Report Service обеспечивает создание жалоб, просмотр собственных жалоб пользователем и обработку жалоб модераторами. Notification Service сохраняет уведомления, предоставляет API для их просмотра и управления ими, а также обеспечивает передачу уведомлений в реальном времени.

Каждый сервис имеет собственную область хранения данных. User Service использует PostgreSQL для учётных записей, ролей, разрешений и журнала аудита. Article Service использует PostgreSQL для дерева материалов, переводов, версий, предложений, поиска и метаданных изображений. Comment Service, Report Service и Notification Service используют собственные базы PostgreSQL для принадлежащих им данных. Межсервисные связи между таблицами разных сервисов не создаются.

Взаимодействие между сервисами организуется двумя способами. Синхронные HTTP-запросы применяются, когда для продолжения операции необходим немедленный ответ другого сервиса. Для передачи событий, связанных с действиями пользователей и изменениями состояния объектов, используется RabbitMQ. Такой механизм позволяет передавать события сервису уведомлений и сервису аудита без непосредственного изменения их баз данных другими сервисами.

RabbitMQ не используется в качестве основного хранилища прикладных данных. Он обеспечивает доставку событий между компонентами системы. Для надёжности предусматриваются устойчивые очереди и постоянные сообщения, подтверждение обработки после записи данных и идемпотентная обработка событий с уникальными идентификаторами.

Redis применяется для хранения токенов обновления, списка блокировок, ограничения частоты запросов и кэширования. Он не заменяет PostgreSQL как основное хранилище структурированных данных и не используется вместо RabbitMQ для передачи событий между сервисами.

Для работы с изображениями Article Service взаимодействует с MinIO. Отдельный микросервис мультимедийных материалов не выделяется: управление изображениями и их метаданными относится к области

ответственности сервиса статей. Загрузка файлов выполняется с использованием предварительно подписанных ссылок, а состояние изображений согласуется с состоянием предложения об изменении статьи.

В качестве механизма взаимодействия клиентского приложения с серверной частью используется HTTP API. Для передачи структурированных данных применяется JSON. Для уведомлений в реальном времени используется SSE через API Gateway. При эксплуатации системы передача данных должна осуществляться по защищённым соединениям.

Микросервисная архитектура позволяет определить владельца данных и отделить функциональные области системы. Вместе с тем она предъявляет дополнительные требования к обработке сетевых ошибок, недоступности сервисов и повторной доставке событий. Поэтому при проектировании необходимо учитывать устойчивость межсервисного взаимодействия, корректность обработки ошибок и независимость баз данных.

Таким образом, серверная часть строится как набор взаимодействующих микросервисов на C++ с использованием Drogon. PostgreSQL применяется для постоянного хранения данных, Redis – для кэширования и вспомогательных механизмов безопасности, RabbitMQ – для передачи событий, а MinIO – для хранения изображений. API Gateway обеспечивает маршрутизацию запросов и выполнение предусмотренных проверок доступа.

2.3 Выбор языка программирования и фреймворка

Для разработки серверных сервисов выбран язык программирования C++20. Он предоставляет средства управления вычислительными ресурсами, многопоточностью и сетевым взаимодействием, а также позволяет использовать внешние библиотеки для создания серверных приложений. Стандарт C++20 включает возможности языка и стандартной библиотеки, применяемые при разработке современных приложений.

В качестве альтернатив могли рассматриваться Java, C# и Python. Все перечисленные языки имеют инструменты для разработки серверных систем. В рамках данного проекта выбран C++, поскольку он соответствует принятому технологическому стеку и позволяет использовать фреймворк Drogon для реализации HTTP-сервисов.

В качестве веб-фреймворка используется Drogon. Он предназначен для разработки серверных приложений на C++ и предоставляет средства маршрутизации HTTP-запросов, формирования ответов, обработки параметров запросов и реализации API. В сочетании с необходимыми библиотеками он позволяет организовать взаимодействие с PostgreSQL и Redis [7].

Каждый микросервис реализуется как отдельное приложение на C++ с использованием Drogon. При этом сервис содержит только логику, относящуюся к его функциональной области. Например, Article Service обрабатывает операции с материалами и предложениями об изменении, а

Comment Service – операции с комментариями. Такое разделение позволяет не объединять логику всех компонентов в одно серверное приложение.

Для работы с PostgreSQL используется SQL непосредственно через DbClient, без применения ORM. Это позволяет явно контролировать выполняемые запросы, структуру выборок, условия изменения данных и транзакционные операции. Такой подход также обеспечивает непосредственное использование возможностей PostgreSQL, включая полнотекстовый поиск и тип ltree для хранения путей в иерархическом дереве.

Обработка HTTP-запроса включает получение и проверку параметров, выполнение соответствующей прикладной логики, обращение к базе данных или другому сервису при необходимости и формирование ответа клиенту. Для внутренних запросов используются определённые API-интерфейсы. События, предназначенные для асинхронной обработки, передаются через RabbitMQ.

Для сборки сервисов используется CMake. Он позволяет описывать исходные файлы, зависимости и параметры компиляции, а также организовать сборку нескольких самостоятельных приложений в рамках одного проекта. Разделение исходного кода по сервисам соответствует выбранной микросервисной архитектуре.

Выбор C++20 и Drogon позволяет реализовать HTTP API, обработку пользовательских запросов, взаимодействие с PostgreSQL и Redis и внутренние интерфейсы микросервисов. Использование SQL через DbClient обеспечивает непосредственный контроль над операциями с данными, а CMake предоставляет единый механизм сборки серверных компонентов.

2.4 Выбор системы управления базами данных

В качестве основной системы управления реляционными базами данных выбрана PostgreSQL. Она используется для постоянного хранения структурированных данных всех микросервисов. При этом каждый сервис владеет собственной базой данных либо отдельной схемой и не обращается напрямую к таблицам других сервисов.

PostgreSQL соответствует требованиям проекта благодаря поддержке реляционной модели, транзакций, ограничений целостности, индексов и полнотекстового поиска. Первичные и внешние ключи, а также ограничения NOT NULL, UNIQUE и CHECK позволяют задавать допустимую структуру данных и контролировать корректность отдельных операций [8].

Особое значение имеет поддержка иерархических данных. Для хранения путей узлов дерева материалов используется расширение ltree. Оно позволяет представлять путь узла в виде последовательности элементов, например witcher.heroes.geralt, и выполнять запросы для получения предков, потомков и поддеревьев. Использование таких путей обеспечивает поддержку иерархической организации материалов и навигации по ней.

PostgreSQL также предоставляет средства полнотекстового поиска. В Article Service для поиска используются типы tsvector и tsquery, соответствующие индексы и функции формирования фрагментов текста с

выделением найденных совпадений. Индексируемое содержимое формируется с учётом языка перевода, а поиск ограничивается актуальными опубликованными версиями материалов.

Транзакции позволяют выполнять связанные изменения данных согласованно. Это важно при обработке предложений об изменении, создании новой версии статьи, изменении состояния предложения и выполнении других операций, затрагивающих несколько записей. Если операция завершается ошибкой, транзакция позволяет не сохранять её частичный результат.

В рамках проекта можно рассматривать MySQL и SQLite как альтернативные СУБД. SQLite отличается простотой развёртывания, однако для системы с несколькими серверными сервисами и одновременной работой пользователей предпочтительнее отдельная серверная СУБД. MySQL также поддерживает реляционную модель, но PostgreSQL предоставляет необходимые проекту средства работы с ltree и полнотекстового поиска, что делает её подходящим выбором для данной предметной области.

Для выполнения SQL-запросов используется DbClient фреймворка Drogon без ORM. Это позволяет непосредственно задавать запросы и контролировать их выполнение, в том числе при использовании транзакций, полнотекстового поиска и операций над иерархическими данными.

Таким образом, PostgreSQL выбрана благодаря поддержке реляционной модели, транзакций, ограничений целостности, индексации, полнотекстового поиска и расширения ltree. Разделение данных по сервисам позволяет закрепить ответственность за каждую область данных и исключить прямой доступ одного сервиса к таблицам другого.

2.5 Выбор средств кэширования и хранения мультимедийных данных

Для выполнения вспомогательных операций с временными данными используется Redis [10]. Он предоставляет структуры данных, хранящиеся в оперативной памяти, и позволяет выполнять операции с ними с небольшой задержкой. В разрабатываемой системе Redis применяется для кэширования, хранения токенов обновления, ведения списка блокировок и ограничения частоты запросов.

Кэширование позволяет уменьшить количество повторных обращений к постоянному хранилищу в случаях, когда результат может быть повторно использован. При этом Redis не является основным хранилищем материалов и пользовательских данных: постоянная структурированная информация сохраняется в PostgreSQL. При использовании кэша необходимо учитывать актуальность сохранённых значений и обновлять или удалять их при соответствующих изменениях данных.

Хранение токенов обновления в Redis позволяет организовать их ротацию и отзыв. При повторном использовании ранее выданного токена система рассматривает это как возможный признак компрометации и закрывает пользовательские сессии в соответствии с принятой логикой. Redis

также используется для хранения идентификаторов заблокированных пользователей, чтобы API Gateway мог отклонять запросы таких пользователей, и для ограничения частоты обращений.

Для передачи событий между микросервисами используется RabbitMQ. Он обеспечивает асинхронную доставку сообщений, например о создании или одобрении предложения, изменении ролей и блокировке пользователя. События используются сервисом уведомлений и сервисом аудита. В отличие от Redis, RabbitMQ выполняет роль брокера сообщений, а не кэша прикладных данных.

Для хранения изображений используется объектное хранилище MinIO, совместимое с S3 API [11]. В PostgreSQL сохраняются метаданные файлов и сведения об их связи с предложениями об изменении или опубликованными переводами. Бинарное содержимое изображений размещается непосредственно в MinIO, что позволяет отделить файловые данные от реляционной структуры.

Обработка изображений относится к ответственности Article Service. Для загрузки используются предварительно подписанные ссылки, позволяющие передавать файл непосредственно в объектное хранилище. Перед загрузкой проверяются допустимость операции, объявленный размер и поддерживаемый формат. Фактический формат определяется по содержимому файла, а не только по расширению. Архитектурой предусмотрены ограничения на размер файла, общий размер изображений в одном предложении и их количество.

Изображения, относящиеся к ожидающим рассмотрения предложениям, размещаются отдельно от опубликованных файлов и доступны автору и модераторам. После одобрения предложения соответствующие изображения переводятся в опубликованное хранилище. При отклонении предложения изображения удаляются, а фоновая задача очищает просроченные и отклонённые файлы. Для изоляции данных используются отдельные контейнеры и отдельные бакеты с учётными данными для соответствующих сервисов; изображения статей принадлежат Article Service, а аватары пользователей – User Service.

Разделение средств хранения позволяет определить назначение каждого компонента. PostgreSQL обеспечивает постоянное хранение структурированных данных, Redis выполняет функции кэширования и поддержки механизмов безопасности, RabbitMQ передаёт события между сервисами, а MinIO хранит бинарные изображения. Такое распределение соответствует принятой микросервисной архитектуре и позволяет каждому компоненту использовать специализированное средство хранения или обмена данными.

3 ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ

.

3.1 Разработка структуры информационной системы

Информационная система проектируется как набор взаимодействующих сервисов с разделением ответственности между ними. В состав системы входят шлюз API (Gateway), сервис пользователей, сервис статей, сервис комментариев, сервис жалоб и сервис уведомлений. Для хранения данных используется PostgreSQL, для кэширования и управления токенами – Redis, для обмена событиями – RabbitMQ, для хранения изображений – MinIO. Развёртывание компонентов предусматривается с помощью Docker Compose.

Gateway является единой точкой входа для клиентских запросов. Он выполняет маршрутизацию, проверяет JWT, обращается к Redis для проверки блокировки пользователя, применяет ограничения частоты запросов и передаёт внутренним сервисам идентификатор пользователя и его роль. Через шлюз также передаётся поток уведомлений, реализованный с помощью Server-Sent Events (SSE).

Сервис пользователей отвечает за регистрацию, аутентификацию, управление профилями, ролями и блокировками, хранение аватаров и ведение журнала действий. Сервис статей реализует работу с деревом материалов, переводами, версиями, предложениями изменений, модерацией, поиском и изображениями статей. Сервис комментариев управляет комментариями и ответами на них. Сервис жалоб хранит обращения пользователей и обеспечивает их рассмотрение. Сервис уведомлений сохраняет уведомления в собственной базе данных и предоставляет API для их получения, прочтения и удаления.

Каждый сервис работает со своей базой данных или схемой PostgreSQL. Прямые связи между таблицами разных сервисов не создаются. Взаимодействие выполняется посредством HTTP-запросов, когда требуется немедленный ответ, и событий RabbitMQ, когда результат должен быть передан другим компонентам без синхронного вызова.

3.2 Разработка концептуальной модели данных

Основу предметной области составляет иерархия материалов об игровых вселенных. Корневые узлы соответствуют игровым вселенным, а дочерние узлы описывают входящие в них сущности. Любой узел может иметь дочерние элементы. Для узлов верхних уровней преимущественно используется общее описание, а подробная информация размещается в конечных узлах дерева.

Узел и его языковые представления разделены. Узел задаёт положение материала в иерархии и идентифицируется собственным идентификатором и slug. Перевод содержит заголовок и текст на определённом языке. Для каждого

перевода сохраняется история версий. Благодаря этому структура материала не зависит от языка, а изменения текста на одном языке не заменяют историю другого перевода.

Работа с материалами включает создание, редактирование, перемещение и удаление. Пользовательские изменения оформляются как предложения, содержащие новую версию текста и ссылку на исходную версию. После проверки предложения модератором изменение либо применяется, либо отклоняется. Если исходная версия успела измениться, предложение признаётся устаревшим и должно быть отправлено повторно на основе актуального текста. Удаление узла оформляется отдельным типом жалобы и допускается только для узла без дочерних элементов. Фактическое удаление выполняется мягко, с сохранением записи и связанной истории.

Комментарии относятся к конкретному переводу, а не к узлу целиком. Ответ на комментарий хранится как дочерний комментарий с указанием родительского идентификатора. Такая организация позволяет строить вложенные обсуждения без ограничения глубины. Удаление комментариев также выполняется мягко.

Концептуальная модель также включает пользователей, роли, разрешения, жалобы, изображения, уведомления и журнал действий. Связи между сущностями разных сервисов отражают логические зависимости и не предполагают создания межсервисных внешних ключей.

Концептуальная модель предметной области приведена в приложении В.

3.3 Разработка логической модели данных

Логическая модель информационной системы определяет состав таблиц, назначение их полей и связи между объектами предметной области. При проектировании выделены данные пользователей, статей, комментариев, жалоб и уведомлений. Каждая группа данных принадлежит отдельному сервису и размещается в собственной базе данных или схеме PostgreSQL.

Внутри базы данных сервиса связи между таблицами обеспечиваются внешними ключами. Идентификаторы объектов, принадлежащих другим сервисам, сохраняются без межсервисных внешних ключей. Проверка таких объектов и согласование связанных операций выполняются через программные интерфейсы и обмен событиями.

База данных сервиса пользователей предназначена для хранения учётных записей, настройки полномочий и регистрации действий. В её состав входят следующие таблицы.

Таблица users хранит сведения об учётных записях пользователей. Поле id является первичным ключом и однозначно идентифицирует пользователя. Поле login содержит уникальное имя, используемое при аутентификации. Поле password_hash хранит хеш пароля, сформированный алгоритмом Argon2, вместе с параметрами, необходимыми для его проверки. Поле role_id является внешним ключом таблицы roles и определяет назначенную пользователю роль. Поле is_banned содержит логический признак блокировки учётной

записи. Поле avatar_url хранит ссылку на изображение профиля, размещённое в объектном хранилище MinIO, и может оставаться незаполненным при отсутствии аватара. Одному пользователю назначается одна глобальная роль, при этом одну роль могут иметь несколько пользователей.

Таблица roles представляет собой справочник глобальных ролей. Поле id является первичным ключом роли. Поле name содержит её уникальное наименование. В системе предусматриваются роли user, moderator, admin и superadmin. Роль определяет уровень полномочий пользователя и участвует в проверке доступности административных операций. Между таблицами roles и users устанавливается связь «один ко многим».

Таблица permissions содержит справочник разрешений на выполнение операций. Поле id является первичным ключом разрешения. Поле name хранит его уникальное наименование, используемое при проверке доступа к функциям системы. Выделение разрешений в отдельную таблицу позволяет определять состав полномочий каждой роли без повторения описаний разрешений в записях пользователей.

Таблица role_permissions реализует связь «многие ко многим» между ролями и разрешениями. Поле role_id является внешним ключом таблицы roles. Поле permission_id является внешним ключом таблицы permissions. Совокупность этих полей образует составной первичный ключ, исключающий повторное назначение одного разрешения одной роли. При аутентификации пользователя разрешения его роли считываются из базы данных и включаются в токен доступа.

Таблица audit_log предназначена для регистрации значимых действий пользователей. Поле event_id содержит уникальный идентификатор события и позволяет предотвращать повторное сохранение записи при повторной доставке сообщения. Поле actor_id определяет пользователя, выполнившего действие. Поле action содержит обозначение операции, например входа в систему, отправки предложения изменения, рассмотрения жалобы или блокировки пользователя. Поле target_id хранит идентификатор объекта, к которому относится действие. Поле occurred_at содержит дату и время события. Поле data предназначено для дополнительных сведений, состав которых определяется видом операции. В частности, при одобрении администратором собственного предложения в дополнительных данных фиксируется соответствующий признак.

Журнал действий формируется на основании событий, получаемых через RabbitMQ. Сведения о чтении статей в журнал не включаются. Доступ к записям ограничивается полномочиями пользователя. Срок хранения составляет по умолчанию 180 дней и может изменяться в конфигурации. Удаление устаревших записей выполняется фоновой задачей.

База данных сервиса статей обеспечивает хранение иерархии материалов, переводов, истории редакций, предложений изменений и метаданных изображений. Она включает следующие таблицы.

Таблица nodes хранит узлы общей иерархии материалов. Поле id является первичным ключом узла. Поле slug содержит строковое обозначение,

используемое при обращении к материалу через программный интерфейс. Поле path определяет положение узла в дереве и в физической модели представляется типом ltree. Поле deleted_at содержит дату и время логического удаления. Отсутствие значения в deleted_at означает, что узел не удалён. Корневые узлы соответствуют игровым вселенным, а дочерние узлы описывают входящие в них объекты. Иерархический путь используется для получения предков, потомков и поддеревьев. Удаление допускается только для узлов, не имеющих дочерних элементов.

Таблица translations содержит сведения о языковых представлениях материалов. Поле id является первичным ключом перевода. Поле node_id является внешним ключом таблицы nodes и определяет узел, к которому относится перевод. Поле language содержит код языка. Сочетание node_id и language должно быть уникальным, поскольку для одного узла предусматривается одно представление на каждом языке. Поле current_version_id содержит ссылку на текущую опубликованную редакцию из таблицы versions. Поле search_vector хранит поисковое представление актуального содержимого и в физической модели имеет тип tsvector. Оно используется при полнотекстовом поиске и обновляется триггером при изменении данных, влияющих на поисковую выдачу. Между таблицами nodes и translations устанавливается связь «один ко многим».

Таблица versions предназначена для хранения истории опубликованных редакций. Поле id является первичным ключом редакции. Поле translation_id является внешним ключом таблицы translations и определяет перевод, которому принадлежит редакция. Поле title содержит заголовок статьи. Поле content хранит полный исходный текст в формате Markdown. Поле author_id содержит идентификатор автора изменений. Поле created_at определяет дату и время создания редакции. Между переводом и редакциями устанавливается связь «один ко многим». Каждая новая опубликованная редакция сохраняется отдельной записью, благодаря чему предыдущие состояния содержимого остаются доступными для просмотра и сравнения. Идентификатор автора сохраняется без внешнего ключа к базе данных сервиса пользователей.

Таблица edit_proposals хранит предложения изменения текста, направленные на рассмотрение модераторам. Поле id является первичным ключом предложения. Поле translation_id содержит ссылку на перевод, для которого подготовлены изменения. Поле base_version_id указывает на редакцию, использованную автором в качестве исходной. Поле new_text хранит полный предлагаемый текст в формате Markdown. Поле author_id содержит идентификатор автора предложения. Поле status_id является внешним ключом таблицы edit_statuses и определяет текущее состояние предложения. Поле moderator_id содержит идентификатор назначенного проверяющего и остаётся незаполненным до назначения. Поле lease_expires_at определяет момент окончания закрепления предложения за проверяющим. Поле created_at содержит дату и время поступления предложения и используется для упорядочивания очереди модерации. Перед одобрением

проверяется, совпадает ли base_version_id с текущей опубликованной редакцией перевода. При несовпадении предложение признаётся устаревшим.

Таблица edit_statuses представляет собой справочник состояний предложений изменений. Поле id является первичным ключом состояния. Поле name содержит его уникальное наименование. Предусматриваются состояния pending – ожидание рассмотрения, approved – одобрение, rejected – отклонение и outdated – утрата актуальности исходной редакции. Между таблицами edit_statuses и edit_proposals устанавливается связь «один ко многим».

Таблица images содержит метаданные изображений, связанных с предложениями изменений и переводами статей. Поле id является первичным ключом изображения. Поле file_key хранит ключ объекта в MinIO. Поле size содержит размер файла в байтах. Поле format определяет фактический формат изображения, установленный по его содержимому. Поле status_id является внешним ключом таблицы image_statuses и задаёт состояние публикации. Поле edit_id содержит ссылку на предложение изменения из таблицы edit_proposals. Поле translation_id содержит ссылку на перевод из таблицы translations. Поля edit_id и translation_id могут оставаться незаполненными в зависимости от этапа обработки изображения; при этом должна обеспечиваться корректная привязка файла к объекту системы. Бинарное содержимое файла хранится в MinIO.

Таблица image_statuses представляет собой справочник состояний изображений. Поле id является первичным ключом состояния. Поле name содержит его уникальное наименование. Предусматриваются состояния ожидания рассмотрения и публикации. Изображения, связанные с ожидающими предложениями, доступны автору и проверяющим. После одобрения предложения изображения переходят в опубликованное состояние. При отклонении предложения связанные файлы удаляются. Между таблицами image_statuses и images устанавливается связь «один ко многим».

Разделение узлов, переводов и редакций позволяет независимо хранить положение материала в иерархии, его языковые представления и историю содержимого. Перемещение узла изменяет его положение в дереве, сохраняя принадлежность переводов и редакций соответствующему материалу.

База данных сервиса комментариев содержит таблицу обсуждений переводов статей.

Таблица comments предназначена для хранения комментариев и ответов. Поле id является первичным ключом комментария. Поле translation_id содержит идентификатор перевода статьи, к которому относится обсуждение. Поле parent_id является внешним ключом этой же таблицы и определяет родительский комментарий. Для комментария верхнего уровня parent_id остаётся незаполненным. Поле author_id содержит идентификатор автора. Поле text хранит текст комментария. Поле created_at определяет дату и время создания записи. Поле deleted_at содержит дату и время логического удаления. Отсутствие значения в deleted_at означает, что комментарий не удалён. Поля

translation_id и author_id относятся к объектам других сервисов, поэтому межсервисные внешние ключи для них не создаются.

Связь таблицы comments с самой собой имеет вид «один ко многим»: один комментарий может иметь несколько непосредственных ответов. Родительский комментарий и ответ должны относиться к одному переводу статьи. Такая организация позволяет формировать обсуждения произвольной глубины. Логическое удаление сохраняет запись, необходимую для поддержания структуры ответов. Признак наличия ответов определяется при получении комментария, а их количество отдельным полем не хранится.

База данных сервиса жалоб обеспечивает регистрацию обращений и хранение результатов их рассмотрения. Она включает следующие таблицы.

Таблица reports предназначена для хранения жалоб и запросов на удаление узлов. Поле id является первичным ключом обращения. Поле type_id является внешним ключом таблицы report_types и определяет вид обращения. Поле status_id является внешним ключом таблицы report_statuses и содержит текущее состояние. Поле target_id хранит идентификатор объекта, к которому относится обращение. Поле reporter_id содержит идентификатор заявителя. Поле reason хранит текст причины обращения. Поле moderator_id определяет назначенного проверяющего и остаётся незаполненным до назначения. Поле lease_expires_at содержит момент окончания закрепления обращения за проверяющим. Поле created_at определяет дату и время создания обращения. Поле resolved_at содержит дату и время принятия окончательного решения и остаётся незаполненным до завершения рассмотрения. Идентификаторы заявителя, проверяющего и целевого объекта сохраняются без внешних ключей к базам данных других сервисов.

Таблица report_types содержит справочник видов обращений. Поле id является первичным ключом типа. Поле name хранит его уникальное наименование. Предусматриваются жалоба на комментарий, жалоба на статью и запрос на удаление узла. Тип обращения определяет смысл идентификатора target_id и порядок обработки решения. Для запроса на удаление узла заполнение поля reason в таблице reports является обязательным. Между таблицами report_types и reports устанавливается связь «один ко многим».

Таблица report_statuses представляет собой справочник состояний обращений. Поле id является первичным ключом состояния. Поле name содержит его уникальное наименование. Предусматриваются состояния new – новое обращение, in progress – рассмотрение, accepted – принятие и rejected – отклонение. Каждое обращение имеет одно текущее состояние, а одному состоянию может соответствовать несколько обращений.

Обращения рассматриваются в порядке поступления. Запись закрепляется за проверяющим на 30 минут. При отсутствии решения в установленный срок обращение возвращается в очередь. После одобрения запроса на удаление узла соответствующую операцию выполняет сервис статей с проверкой отсутствия дочерних узлов.

База данных сервиса уведомлений предназначена для сохранения сообщений о событиях системы и учёта их прочтения. Она включает следующие таблицы.

Таблица notifications хранит уведомления пользователей. Поле id является первичным ключом уведомления. Поле recipient_id содержит идентификатор получателя. Поле type_id является внешним ключом таблицы notification_types и определяет вид уведомления. Поле object_id содержит идентификатор связанного объекта, например предложения изменения, обращения или комментария. Поле data хранит дополнительные сведения, необходимые для формирования сообщения, в том числе название материала или результат рассмотрения. Поле is_read содержит логический признак прочтения. Поле created_at определяет дату и время создания уведомления. Поля recipient_id и object_id относятся к объектам других сервисов и сохраняются без межсервисных внешних ключей. Готовый текст сообщения формируется при выдаче на основании типа уведомления и дополнительных данных.

Таблица notification_types представляет собой справочник видов уведомлений. Поле id является первичным ключом типа. Поле name содержит его уникальное наименование. Предусматриваются уведомления об одобрении, отклонении и устаревании предложения изменения, завершении рассмотрения обращения, блокировке пользователя и получении ответа на комментарий. Между таблицами notification_types и notifications устанавливается связь «один ко многим».

Уведомления создаются при обработке событий RabbitMQ. Сохранённые записи используются для получения списка сообщений и восстановления пропущенных уведомлений после повторного подключения клиента к SSE-потоку.

При проектировании модели выполняется разделение данных с учётом функциональных зависимостей. Наименования ролей и разрешений хранятся в соответствующих справочниках, а записи пользователей содержат идентификаторы ролей. Благодаря этому изменение наименования роли выполняется в одной записи и не требует обновления связанных пользователей. В таблице role_permissions оба поля входят в состав первичного ключа; атрибуты, зависящие только от роли или только от разрешения, в неё не включаются.

Разделение материалов на узлы, переводы и редакции устраняет повторное хранение общих сведений. Положение материала в дереве определяется узлом, язык представления – переводом, заголовок и текст конкретной редакции – записью versions. Изменение текста одного перевода не затрагивает историю других языковых представлений. Предложения изменений выделяются в отдельную таблицу, поскольку имеют собственные состояния и сведения о рассмотрении.

Состояния предложений, изображений и обращений, а также типы обращений и уведомлений представлены справочниками. Основные таблицы содержат ссылки на соответствующие записи. Это обеспечивает единообразие

допустимых значений и позволяет централизованно управлять их наименованиями.

Поисковое представление search_vector является осознанно сохраняемым производным значением. Оно формируется на основании языка перевода и содержимого текущей опубликованной редакции. Такое решение ускоряет полнотекстовый поиск, но требует поддержания согласованности с исходными данными. Обновление поискового представления выполняется триггером при изменении данных, влияющих на его содержимое.

Использование сохранённых путей ltree позволяет получать поддеревья и цепочки предков средствами PostgreSQL. При этом перемещение узла требует обновления путей его потомков. Эти изменения должны выполняться в одной транзакции с проверкой запрета на перемещение узла внутрь собственного поддерева.

Ссылка current_version_id должна указывать на редакцию, принадлежащую тому же переводу. Аналогично исходная редакция предложения должна соответствовать переводу, для которого подготовлены изменения. Проверка этих правил дополняет ограничения ссылочной целостности и предотвращает создание логически некорректных связей.

Дополнительные сведения событий и уведомлений имеют переменный состав, поэтому сохраняются в полях data. Постоянные атрибуты, используемые для установления связей, определения состояния и фильтрации записей, представлены самостоятельными полями. Такая организация сохраняет определённую структуру основных объектов и позволяет передавать дополнительные сведения, характерные для отдельных видов событий.

Логическая модель данных с распределением сущностей по сервисам приведена в приложении Г.

3.4 Разработка физической модели базы данных

Физическая модель определяет типы столбцов PostgreSQL, ограничения целостности, индексы и специальные механизмы обработки данных. При реализации используется C++20 и фреймворк Drogon. Доступ к базе данных осуществляется через DbClient с выполнением SQL-запросов без применения ORM. Структура таблиц должна соответствовать требованиям третьей нормальной формы.

Для хранения иерархии материалов используется расширение PostgreSQL ltree. В таблице узлов хранится путь, отражающий положение элемента в дереве. Поиск потомков, получение предков для построения навигационной цепочки и перемещение ветвей выполняются с использованием условий по пути. При перемещении узла необходимо запретить перенос внутрь собственного поддерева. После изменения положения обновляется путь перемещённого узла и его потомков.

Таблица переводов содержит языковой код, идентификатор текущей версии и поле типа tsvector для полнотекстового поиска. Поисковое

представление обновляется триггером. Конфигурация полнотекстового поиска определяется языком перевода, а выдача формируется только по опубликованным текущим версиям. Для формирования фрагментов найденного текста используется функция ts_headline. Поиск может выполняться по всей базе материалов или ограничиваться заданной ветвью дерева.

Для каждого предложения сохраняется идентификатор исходной версии перевода. Перед применением проверяется, остаётся ли эта версия текущей. Если версия устарела, предложение не применяется; после одобрения другого предложения ожидающие изменения того же перевода также признаются устаревшими.

Модерация реализуется с использованием блокировки строк PostgreSQL и конструкции FOR UPDATE SKIP LOCKED. Она позволяет выдавать элемент очереди только одному проверяющему, не блокируя обработку остальных элементов. Назначение действует 30 минут. Если решение за это время не принято, элемент возвращается в очередь. Очередь обрабатывается от наиболее старых элементов к новым.

Для мягкого удаления в соответствующих таблицах предусматриваются поля времени удаления. Это позволяет сохранять историю материалов и комментариев, а также связанные жалобы и записи журнала. Очистка журнала действий выполняется фоновой задачей: записи старше установленного срока хранения, составляющего по умолчанию 180 дней, удаляются ежедневно.

Пароли пользователей сохраняются только в виде хешей Argon2. Access-токен JWT действует 15 минут, а refresh-токен – 30 дней и хранится в Redis. При обновлении токенов используется ротация: старый refresh-токен становится недействительным. Повторное использование уже заменённого токена рассматривается как возможная компрометация и приводит к завершению всех сессий пользователя.

Физическая модель базы данных приведена в приложении Д.

3.5 Проектирование API и взаимодействия сервисов

Взаимодействие сервисов организуется через HTTP API для синхронных операций и RabbitMQ для асинхронной передачи событий.

Создание и изменение материалов выполняется через сервис статей. Он принимает предложение, сохраняет его исходную версию и вычисляет построчную разницу между исходным и новым текстом. Для построения diff используется алгоритм Майерса. Просматривать предложения могут их автор, модераторы и администраторы. Пользователь не может самостоятельно опубликовать изменение: оно должно пройти модерацию.

Сервис жалоб обрабатывает пользовательские обращения и предоставляет очередь их рассмотрения. Запрос на удаление узла является специальным типом жалобы с обязательным пояснением. После одобрения такого обращения сервис статей выполняет мягкое удаление узла. Очереди предложений изменений и жалоб реализованы отдельно в соответствующих

сервисах, но используют одинаковый принцип назначения проверяющего и ограничения времени обработки.

Для обмена событиями используется RabbitMQ с общим topic exchange. Тип события задаёт ключ маршрутизации. Сообщение содержит уникальный идентификатор, тип, время, сведения об инициаторе и объекте действия, а также дополнительные данные. Очереди подписчиков являются долговечными, сообщения сохраняются, а подтверждение обработки отправляется после записи результата в базу данных. Уникальный идентификатор события используется для идемпотентной обработки повторных сообщений.

Сервис уведомлений получает события от других сервисов, сохраняет уведомления в собственной базе и передаёт их пользователям через SSE. Клиент устанавливает длительное соединение с Gateway, который проверяет токен и передаёт поток от сервиса уведомлений. При повторном подключении клиент указывает идентификатор последнего полученного уведомления, что позволяет передать пропущенные записи из базы данных. Gateway закрывает поток при блокировке пользователя или истечении срока действия токена.

Журнал действий хранится в сервисе пользователей и заполняется на основании событий RabbitMQ. В нём фиксируются значимые пользовательские и административные операции, а чтение статей не регистрируется. Доступ к журналу определяется ролью и уровнем полномочий.

3.6 Проектирование пользовательских сценариев и обработки данных

Основной сценарий изменения материала начинается с выбора узла и перевода. Пользователь отправляет предложение с полным новым текстом, после чего сервис статей сохраняет его и формирует сравнение с исходной версией. Предложение помещается в очередь модерации и становится доступным автору и пользователям с соответствующими полномочиями.

Модератор получает следующий элемент очереди. Система закрепляет его за проверяющим на 30 минут. Перед одобрением проверяется актуальность исходной версии. Если версия не изменилась, предложение может быть принято, после чего новая версия становится текущей, а связанные изображения переводятся в опубликованное состояние. Если исходная версия устарела, предложение не применяется, автор получает уведомление и должен подготовить изменение на основе актуального текста. При отклонении предложения связанные с ним неопубликованные изображения удаляются.

Изображения загружаются непосредственно в MinIO по предварительно подписанным URL. Размер одного файла ограничен 20 МБ, общий объём изображений одного изменения – 100 МБ, количество файлов – двадцатью. Поддерживаются PNG, JPEG и WebP; формат определяется по содержимому файла. Ограничения должны задаваться конфигурацией. До одобрения предложения изображения доступны только автору и модераторам.

Сценарий обсуждения статьи предусматривает создание комментариев и вложенных ответов. При загрузке сначала запрашиваются комментарии верхнего уровня, а ответы к отдельному комментарию загружаются отдельным запросом. Выдача выполняется порциями с использованием курсора, без нумерации страниц. Комментарии и ответы сортируются от старых к новым. Пользователь может редактировать и удалять собственные комментарии, а модератор – удалять любой комментарий. Удаление выполняется без физического удаления записи.

При создании жалобы пользователь указывает объект и причину обращения. Жалоба получает статус и помещается в очередь рассмотрения. После решения её статус обновляется, а пользователь получает уведомление. Для запросов на удаление узлов дополнительно проверяется наличие дочерних элементов; узел с дочерними элементами удалить нельзя.

Поиск выполняется по опубликованным текущим версиям переводов. Пользователь может искать материалы во всей системе или ограничить область поиска определённой ветвью дерева. Результаты содержат фрагменты текста, сформированные средствами PostgreSQL. Если перевод на выбранном языке отсутствует, система сообщает об этом и показывает доступные языки материала.

Операции с ролями и блокировками выполняются с учётом иерархии полномочий. Администратор может назначать и снимать роль модератора, а суперпользователь – роли администратора и модератора. Назначение ролей и блокировка допускаются только для пользователей, находящихся строго ниже инициатора в иерархии. При блокировке идентификатор пользователя помещается в Redis, а Gateway отклоняет его запросы независимо от того, остаётся ли действительным ранее выданный токен.

Таким образом, проектирование определяет структуру сервисов, распределение данных и порядок обработки ключевых операций. Концептуальная, логическая и физическая модели отражают один состав предметных сущностей на разных уровнях детализации, а API и событийное взаимодействие обеспечивают выполнение сценариев редактирования, модерации, обсуждения и уведомления пользователей.

4 РАЗРАБОТКА ИНФОРМАЦИОННОЙ СИСТЕМЫ

.

4.1 Реализация базы данных

Для хранения данных информационной системы используются PostgreSQL и отдельные базы данных для сервисов. Такое разделение исключает прямые внешние ключи между таблицами разных сервисов и позволяет каждому сервису управлять собственными данными.

База данных User Service содержит сведения о пользователях, ролях, разрешениях и журнале аудита. Данные пользователей включают логин, хеш пароля, роль, признак блокировки и ссылку на аватар. Связь ролей с разрешениями реализуется через отдельные таблицы.

База данных Article Service предназначена для хранения структуры статей и их содержимого. Таблица узлов содержит идентификатор, уникальный идентификатор в виде slug, путь в иерархии и время удаления. Для хранения иерархических путей применяется расширение ltree. Оно позволяет получать дочерние узлы, предков и поддеревья с помощью запросов по префиксу пути.

Таблица переводов связывает узел с языком и текущей версией текста. История версий хранится отдельно: каждая запись содержит заголовок, текст в формате Markdown, автора и время создания. Предложения изменений также хранятся отдельно и включают идентификатор перевода, базовую версию, новый текст, автора, статус и сведения о модерации.

В базе данных Comment Service хранятся комментарии, идентификаторы переводов, авторы, ссылки на родительские комментарии и даты создания или удаления. Вложенность ответов реализуется через поле parent_id. База данных Report Service содержит жалобы, типы и статусы обращений, причину, автора и сведения о рассмотрении. Notification Service хранит уведомления, получателей, типы событий, данные связанных объектов, признаки прочтения и время создания.

Для ускорения поиска используется полнотекстовый индекс PostgreSQL. Поле tsvector формируется на основе опубликованной текущей версии перевода и обновляется триггером при изменении данных. Для обеспечения целостности используются первичные и уникальные ключи, ограничения и внешние ключи в пределах одной базы данных.

4.2 Реализация серверной части

Серверная часть разрабатывается на C++20 с использованием фреймворка Drogon. Доступ к PostgreSQL осуществляется через DbClient с выполнением SQL-запросов без применения ORM.

Система разделена на Gateway и специализированные сервисы. Gateway принимает внешние HTTP-запросы, проверяет JWT, применяет ограничения частоты запросов, проверяет блокировку пользователя через Redis и передаёт запрос соответствующему сервису. Идентификатор пользователя и его роль

передаются внутренним сервисам в составе доверенных служебных заголовков.

User Service отвечает за регистрацию, вход, выход, управление профилем, ролями и блокировками, а также ведение журнала аудита. Article Service реализует работу с узлами и переводами, версиями статей, предложениями изменений, модерацией, поиском и изображениями статей. Comment Service обеспечивает создание, редактирование и удаление комментариев. Report Service обрабатывает жалобы и запросы на удаление узлов. Notification Service хранит уведомления, предоставляет операции их просмотра и изменения состояния, а также передаёт новые уведомления клиентам через SSE.

Для взаимодействия сервисов используются синхронные HTTP-запросы и события RabbitMQ. HTTP применяется, когда вызывающему сервису требуется немедленный результат. RabbitMQ используется для передачи событий, связанных с действиями пользователей, модерацией, уведомлениями и аудитом.

Для обмена событиями применяется единый формат, включающий идентификатор события, его тип, время создания, сведения об инициаторе, целевом объекте и дополнительные данные. Сообщения сохраняются в очередях, а подтверждение обработки отправляется после записи результата в базу данных. Повторная доставка обрабатывается с учётом уникального идентификатора события, чтобы исключить повторное выполнение одного действия.

Сервисы и инфраструктурные компоненты запускаются в контейнерах Docker и объединяются с помощью Docker Compose.

4.3 Реализация аутентификации и разграничения доступа

Аутентификация выполняется через единую форму входа для всех ролей. После успешной проверки логина и пароля сервер выдаёт JWT-токен доступа и токен обновления. Пароли хранятся в виде хешей, рассчитанных алгоритмом Argon2.

Токен доступа действует 15 минут и содержит идентификатор пользователя и роль. Gateway проверяет токен и передаёт сведения о пользователе внутренним сервисам, которые дополнительно проверяют полномочия для соответствующих операций. Токен обновления действует 30 дней и хранится в Redis. При обновлении токена выполняется ротация: старый токен становится недействительным, а клиент получает новый. Повторное использование старого токена рассматривается как возможная компрометация и приводит к закрытию всех сессий пользователя.

При выходе из системы токен обновления удаляется из Redis. При блокировке пользователя его идентификатор помещается в список блокировок Redis, а токены обновления аннулируются. Gateway проверяет наличие пользователя в списке блокировок до передачи запроса внутреннему сервису.

В системе предусмотрены роли superadmin, admin, moderator и user. Пользователь может выполнять только те действия, которые разрешены его ролью. Администраторы управляют модераторами, суперадминистратор может управлять администраторами и модераторами. Назначение ролей и блокировка доступны только для пользователей с более низким уровнем прав. Суперадминистратор создаётся при первом запуске системы на основании переменных окружения.

Изменения ролей и разрешений не обновляют уже выданные токены немедленно: новые права применяются после истечения текущего токена доступа, то есть в пределах 15 минут. Проверка полномочий дополнительно выполняется в сервисах, отвечающих за соответствующие операции.

4.4 Реализация механизма редактирования и версионирования статей

Изменения опубликованных статей проходят через механизм предложений и модерации. Пользователь отправляет новую версию текста, которая сохраняется как предложение изменения и становится доступной автору, модераторам и администраторам. До одобрения предложения опубликованная версия статьи не изменяется.

Каждое предложение содержит идентификатор базовой версии, на основе которой оно было создано. Для отображения различий между опубликованным и предложенным текстом используется построчный алгоритм Myers diff. Это позволяет модератору просматривать добавленные и удалённые строки.

Перед одобрением система проверяет, совпадает ли базовая версия предложения с текущей опубликованной версией перевода. Если за это время статья была изменена, предложение получает статус устаревшего. Пользователь уведомляется о необходимости повторно отправить изменения на основе актуального текста. Автоматическое трёхстороннее объединение изменений не выполняется.

После одобрения предложения создаётся новая версия перевода, которая становится текущей. Другие ожидающие предложения для того же перевода помечаются как устаревшие. При отклонении предложения опубликованный текст остаётся без изменений. Предложения обрабатываются в порядке поступления.

Для предотвращения одновременного рассмотрения одного предложения применяется блокировка записи при выборе задачи из очереди модерации. Задача закрепляется за модератором на 30 минут. Если решение не принято за это время, задача возвращается в очередь. Модератор не может рассматривать собственные изменения; для администратора и супер администратора допускается рассмотрение собственных предложений с обязательной записью этого действия в аудит.

Перемещение узлов дерева также проходит модерацию. Система запрещает перемещать узел внутрь собственного поддерева. Удалять можно

только конечные узлы; удаление выполняется логически через заполнение поля deleted_at. Запрос на удаление оформляется как отдельный тип обращения с обязательным указанием причины. После одобрения такого запроса Article Service выполняет удаление узла.

4.5 Реализация работы с Redis

Redis используется для хранения токенов обновления, списка заблокированных пользователей и данных, необходимых для ограничения частоты запросов. Он не применяется в качестве брокера сообщений: передача событий между сервисами выполняется через RabbitMQ.

При обновлении токена прежняя запись аннулируется и создаётся новая. При выходе пользователя или блокировке его сессия становится недействительной за счёт удаления соответствующих токенов.

Список заблокированных пользователей позволяет Gateway проверять статус пользователя до обращения к внутренним сервисам. Это обеспечивает централизованную проверку блокировки и не требует выполнения отдельного запроса к базе данных пользователей для каждого входящего обращения.

Redis также используется Gateway для ограничения частоты запросов. Это позволяет ограничивать количество обращений за определённый интервал времени и снижать риск чрезмерной нагрузки на серверные сервисы. Конкретные значения ограничений задаются конфигурацией системы.

4.6 Реализация работы с мультимедийными материалами

Для хранения изображений используется MinIO. Аватары пользователей размещаются в хранилище User Service, а изображения статей и предложений изменений – в хранилище Article Service. Для сервисов используются отдельные корзины и учётные данные доступа.

Загрузка файлов выполняется по предварительно подписанным URL. Перед загрузкой клиент сообщает размер файла, после чего система проверяет соблюдение установленных ограничений. Максимальный общий размер изображений в одном изменении составляет 100 МБ, размер отдельного файла – 20 МБ, количество файлов в одном изменении – не более 20.

Поддерживаются форматы PNG, JPEG и WebP. Формат определяется по содержимому файла, а не по его расширению. Ограничения размера применяются также на стороне объектного хранилища, чтобы предотвратить загрузку файлов, превышающих допустимый объём.

Изображения, прикреплённые к предложению изменения, до принятия решения хранятся в области ожидающих файлов и доступны автору предложения и сотрудникам, участвующим в модерации. При одобрении предложения изображения переводятся в публичную область хранения. При отклонении предложения соответствующие файлы удаляются. Дополнительная задача очистки удаляет просроченные и отклонённые файлы.

Изображения рассматриваются как часть предложения изменения и отдельно не модерируются. Поэтому публикация нового текста и связанных с ним изображений выполняется в рамках одного процесса принятия решения.

5 ТЕСТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ

.

5.1 Разработка сценариев тестирования

5.2 Тестирование основных функций

5.3 Тестирование базы данных

5.4 Тестирование механизмов версионирования и разграничения доступа

5.5 Анализ результатов тестирования

6 РУКОВОДСТВО ПО РАЗВЁРТЫВАНИЮ ПРИЛОЖЕНИЯ

.

6.1 Требования к программному и аппаратному обеспечению

6.2 Настройка окружения и конфигурации системы

6.3 Развёртывание и запуск информационной системы

6.4 Проверка работоспособности после запуска

7 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

.

7.1 Начало работы с системой

7.2 Регистрация и аутентификация пользователей

7.3 Навигация по игровым вселенным и поиск статей

7.4 Создание и редактирование статей

7.5 Работа с комментариями и жалобами

7.6 Просмотр уведомлений и управление профилем

ЗАКЛЮЧЕНИЕ

СПИСОК ЛИТЕРАТУРНЫХ ИСТОЧНИКОВ

[1] Доманов, А. Т. Стандарт предприятия / А. Т. Доманов, Н. И. Сорока. – Минск: БГУИР, 2024. – 167 с.

[2] PostgreSQL. Full Text Search [Электронный ресурс]. – Режим доступа: https://www.postgresql.org/docs/17/textsearch.html. – Дата доступа: 29.09.2026.

[3] Fandom. [Электронный ресурс]. – Режим доступа: https://help.fandom.com/wiki/Gamepedia. – Дата доступа: 28.09.2026.

[4] Wiki.gg [Электронный ресурс]. – Режим доступа: https://www.wiki.gg/. – Дата доступа: 28.09.2026.

[5] MediaWiki [Электронный ресурс]. – Режим доступа: https://www.mediawiki.org/wiki/Miraheze. – Дата доступа: 28.09.2026.

[6] PostgreSQL. Documentation [Электронный ресурс]. – Режим доступа: https://www.postgresql.org/docs/17/. – Дата доступа: 29.09.2026.

[7] Drogon. Drogon Web Framework [Электронный ресурс]. – Режим доступа: https://github.com/drogonframework/drogon. – Дата доступа: 29.09.2026.

[8] PostgreSQL. Constraints [Электронный ресурс]. – Режим доступа: https://www.postgresql.org/docs/17/ddl-constraints.html. – Дата доступа: 29.09.2026.

[9] PostgreSQL. Triggers [Электронный ресурс]. – Режим доступа: https://www.postgresql.org/docs/current/triggers.html. – Дата доступа: 29.09.2026.

[10] Redis. Redis data types [Электронный ресурс]. – Режим доступа: https://redis.io/docs/latest/develop/data-types/. – Дата доступа: 29.09.2026.

[11] Amazon Web Services. Amazon Simple Storage Service Documentation [Электронный ресурс]. – Режим доступа: https://docs.aws.amazon.com/s3/. – Дата доступа: 29.09.2026.

ПРИЛОЖЕНИЕ А
(обязательное)
Справка о проверке на заимствование

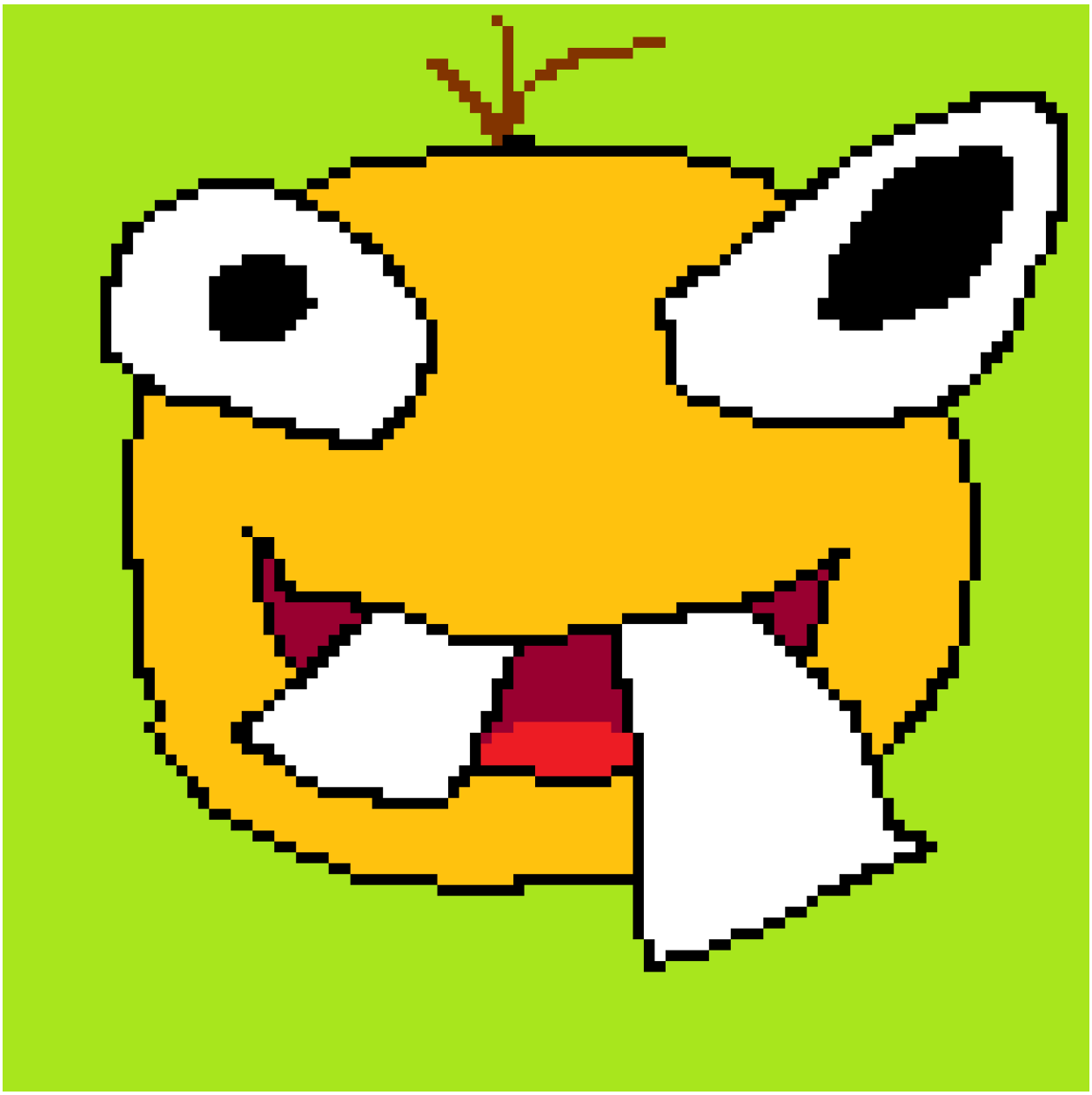


Рисунок А.1 – Справка о проверке на заимствование

ПРИЛОЖЕНИЕ Б
(обязательное)
Листинг программного кода

sddasds

ПРИЛОЖЕНИЕ В
(обязательное)
Концептуальная модель

ПРИЛОЖЕНИЕ Г
(обязательное)
Логическая модель

ПРИЛОЖЕНИЕ Д
(обязательное)
Физическая модель

ПРИЛОЖЕНИЕ Е
(обязательное)
Ведомость документов